

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING...

Working with Classes



RECAP LAST WEEK – Q1

Which of the following programming language(s) is/are considered as Object-Oriented programming languages?

1. Python
2. C
3. Java
4. Pascal
5. Basic

RECAP LAST WEEK — Q1 ANSWER

Which of the following programming language(s) is/are considered as Object-Oriented programming languages?

1. **Python**
2. C
3. **Java**
4. Pascal
5. Basic

RECAP LAST WEEK — Q2

In “Procedural Oriented Programming”,

1. adding new data and function is easy.
2. there is NO proper way for hiding data, so it is less secure.
3. overloading is possible.
4. program is divided into small parts called objects.
5. data is more important than functions.

RECAP LAST WEEK — Q2 ANSWER

In “Procedural Oriented Programming”,

1. adding new data and function is easy.
2. **there is NO proper way for hiding data, so it is less secure.**
3. overloading is possible.
4. program is divided into small parts called objects.
5. data is more important than functions.



DATA TYPES

DATA TYPES

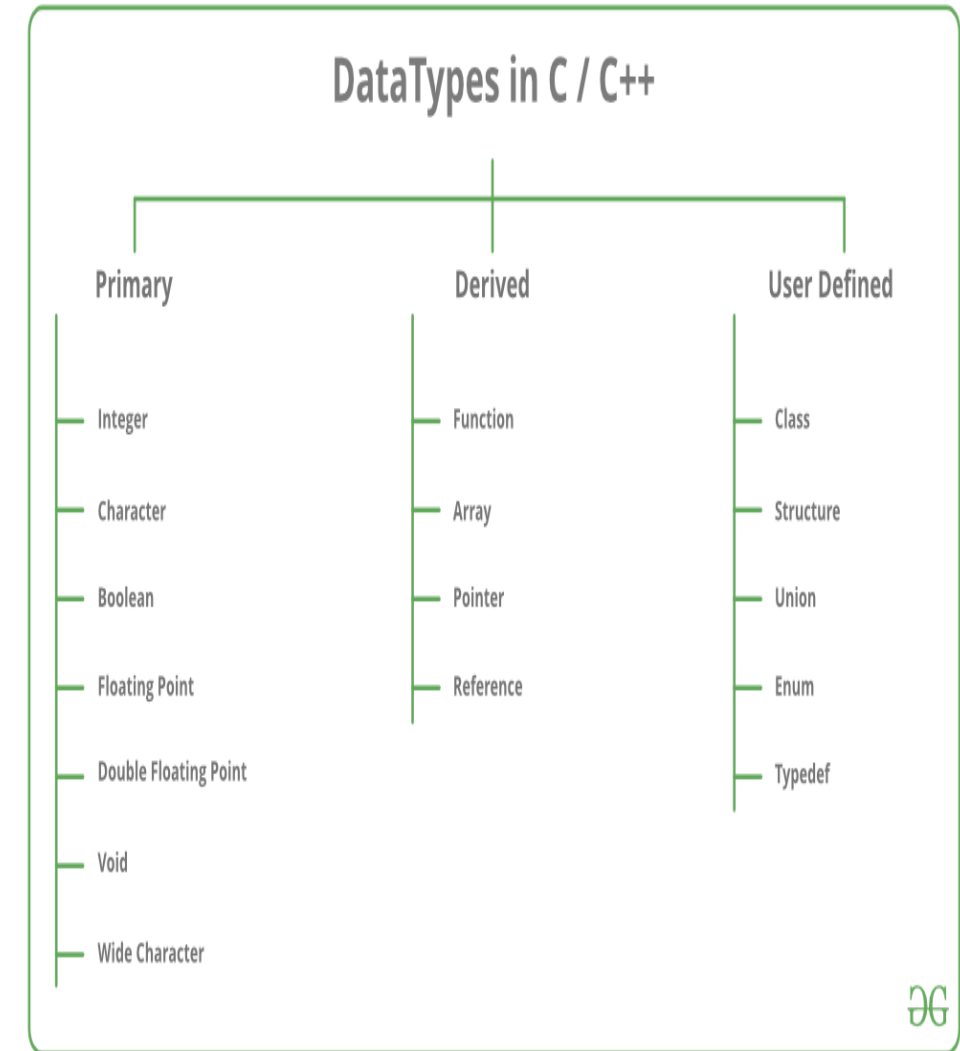
- The type of data the variables can store.
- The compiler allocates some memory for the variable at the declaration based on the data-type.
- Every data type requires a different amount of memory.

Example:

- Integer: Keyword (int) - Requires 4 bytes of memory space and ranges from -2147483648 to 2147483647.
- Character: Keyword (char) - Requires 1 byte of memory space and ranges from -128 to 127 or 0 to 255.

DATA TYPES

- **Build-in/Primitive Data Types:**
 - Built-in or predefined data types
 - Can be used directly by the user to declare variables.
 - Example: int, char , float, bool, void etc.
- **Derived Data Types:**
 - Derived from the primitive or built-in datatypes
 - Example: Function, Array, Pointer, Reference
- **User-Defined Data Types:**
 - Defined by user.
 - Example: Class, Structure, Enum, Typedef



DERIVED DATA TYPES

Derived from the primitive or built-in datatypes.

- **Function** - A block of code or program segment that is defined to perform a specific well-defined task. All the lines of code are put together inside a single function and this can be called anywhere required

Syntax: ReturnType FunctionName(parameters)

- **Array** - A collection of items stored at continuous memory locations. Represent many instances in one variable

Syntax: DataType ArrayName[size_of_array];

- **Pointers** - A symbolic representation of addresses. A pointer points to an address which holds the data

*Syntax: datatype *var_name;*

- **References** - An alternative name for an existing variable. Declared as a reference by putting '&'

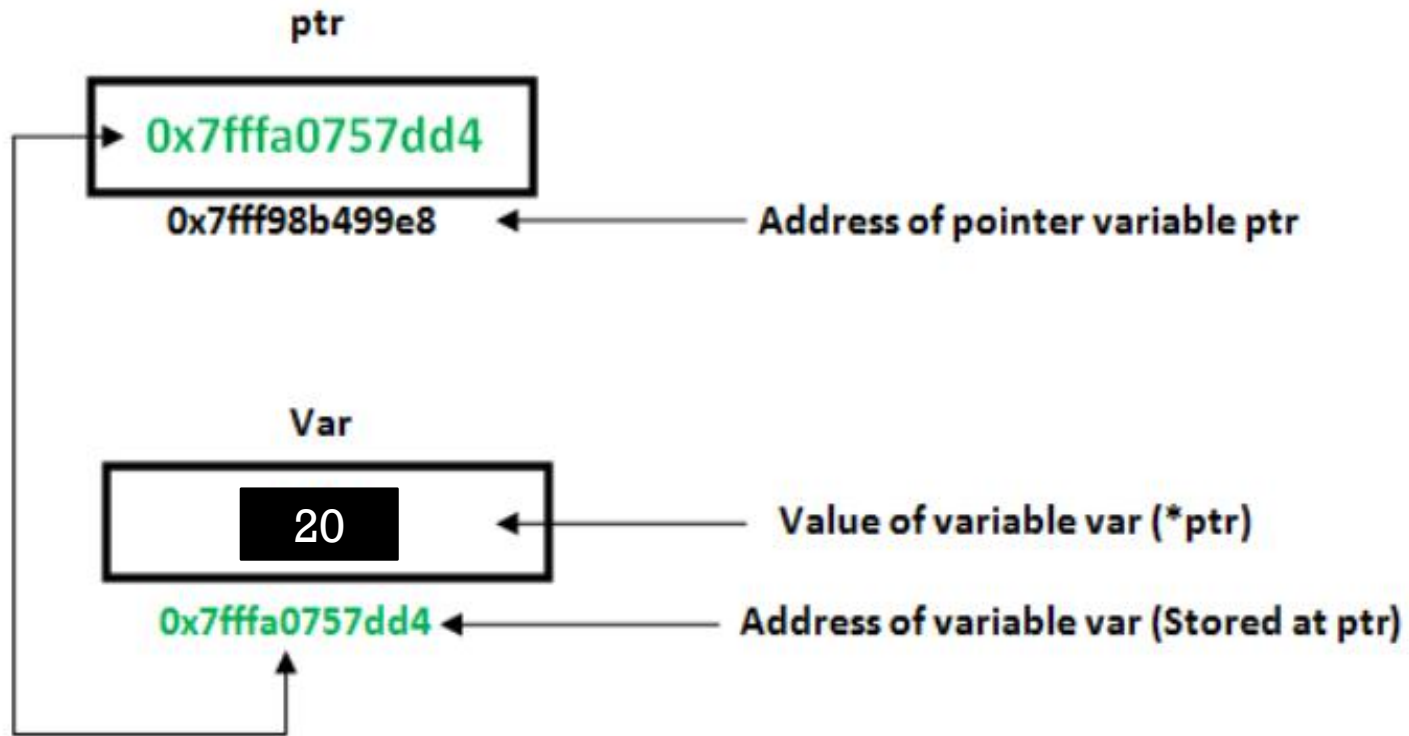
ACTIVITY:

WRITE THE OUTPUT

```
Value at ptr = 0x6ffdfc
Value at var = 20
Value at *ptr = 20
20
Value of var = 40
```

```
-----
Process exited after 0.2104 seconds with return value 0
Press any key to continue . . . |
```

```
1  #include<iostream>
2  using namespace std;
3
4  void pointers()
5  {
6      int var = 20;
7      int* ptr;
8      ptr = &var;
9      cout << "Value at ptr = " << ptr << "\n";
10     cout << "Value at var = " << var << "\n";
11     cout << "Value at *ptr = " << *ptr << "\n";
12
13     int &ref=var;
14     cout << ref << endl;
15     ref = 40;
16     cout<< "Value of var = " <<var<<endl;
17 }
18
19 int main()
20 {
21     pointers();
22     return 0;
23 }
```



```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int i = 10; // simple or ordinary variable.
7      int* p = &i; // single pointer
8      int** pt = &p; // double pointer
9      int*** ptr = &pt; // triple pointer
10
11     cout << "i = " << i << "\t"
12          << "p = " << p << "\t"
13          << "pt = " << pt << "\t"
14          << "ptr = " << ptr << '\n';
15
16     int a = 5;
17     int& S = a;
18     int& S0 = S;
19     int& S1 = S0;
20
21     cout << "a = " << a << "\t"
22          << "S = " << S << "\t"
23          << "S0 = " << S0 << "\t"
24          << "S1 = " << S1 << '\n';
25     return 0;
26 }

```

USER-DEFINED DATA TYPES

- Variables of simple data types (float, char, and int) represent one item of information.
 - float: Price of sugar, char: a character, int: number of students
- But just as groceries are organized into bags, employees into departments, and words into sentences.....
- Organize simple variables into more complex entities.
- In C/C++ we define *structure to get this done*.



EXERCISE

Write a simple C structure to store details of a student (Name, age, height & weight) and print these student details on the console

```
#include <stdio.h>
```

```
struct Student {  
    char name[50];  
    int age;  
    float height;  
    float weight;  
};
```

```
int main() {  
    struct Student s1 = {"Mala", 20, 175.5, 65.0};  
    printf("Name: %s\n", s1.name);  
    printf("Age: %d\n", s1.age);  
    printf("Height: %.2f cm\n", s1.height);  
    printf("Weight: %.2f kg\n", s1.weight);  
    return 0;  
}
```

STRUCTURES

- A user-defined data type in C/C++
- Store a group of data (similar data types or non-similar data types)
- Creates a data type that can be used to group items of possibly different types into a single type
- The 'struct' keyword is used to create a structure.
- Structure members can be initialized with declaration in C++
- Contain two types of members:
 - Data Member: Variables of different/similar data types in C++.
 - Member Functions: Can include functions inside a structure declaration.

```

1  #include<iostream>           //include the header file
2  using namespace std;        //use names for objects and variables
3
4  struct Student{
5      string name = " ";
6      int age =0;
7      float height = 0.0;
8      float weight = 0.0;
9  };
10
11 int main()
12 {
13     Student S1;
14     S1.name="Mala";
15     S1.age=10;
16     S1.height=7.5;
17     S1.weight=23;
18
19     cout<<S1.name<<endl;
20     cout<<S1.age<<endl;
21     cout<<S1.height<<endl;
22     cout<<S1.weight<<endl;
23 }

```

STRUCTURES

- A structure is an aggregate of data types built using elements of other types

```
struct date{  
    int day;  
    char month[10];  
    int year;  
};
```

- Each structure definition must end with a semicolon ; (structure termination)

- The data items in a structure are called the *members* of the structure.
- Has three members:
 - *day*-an integer,
 - *month*-a character array,
 - *year*-an integer.

Declare a structure variable called 'today'

```
date today;
```

STRUCTURE SYNTAX

Structure Name

Creates a new data type

struct date{

int day;

char month[10];

int year;

} ;

Structure members

Structure Termination

STRUCTURES

- Defining a Structure Variable
 - Structure variables are declared like variables of other types
 - Example:
`date today, dateArray[10], *datePtr, &dateRef = dateObject;`
 - Arrays of structures can also be defined.
`date dateArray[10];`
 - The variable name must be used to distinguish one variable from another (date1, date2)
- In C - need to include the keyword **struct** in defining structure variables. But in C++ it is optional.

Example C

`Struct date today;` (In C++ the keyword **struct** is not necessary)

ACCESSING MEMBERS OF THE STRUCTURE

- Member access operators:
 - Dot operator (.)
- Syntax
Name of the structure variable.member name;
- Structure members are treated just like other variables.

[ACTIVITY] Print member day of the variable 'today'

```
cout << "Day" << today.day << endl;
```

EXERCISE 1: INITIALIZING STRUCTURE VARIABLES

```
#include<iostream>

using namespace std;

//defining the structure date

struct date{

    int day;

    char month[10];

    int year;

}; // end of structure definition

int main()

{

    date today={6,"May",2026};

    cout<<"Day:"<<today.day

        <<" Month:"<<today.month

        <<" Year:"<<today.year<<endl;

}
```

ACTIVITY

```
1  #include<iostream>
2  #include<string.h>
3
4  using namespace std;
5
6  //defining the structure date
7  struct date{
8      int day;
9      char month[10];
10     int year;
11
12     void displayDate(){
13         cout<<"Day:"<<day
14         <<" Month:"<<month
15         <<" Year:"<<year<<endl;
16     }
17 }yesterday,d1; // end of structure definition
```

```
20 int main()
21 {
22     yesterday.day=5;
23     strcpy(yesterday.month,"May");
24     yesterday.year=2026;
25
26     d1=yesterday;
27
28     cout<<"YESTERDAY"<<endl;
29     yesterday.displayDate();
30
31     //Display values of d1
32     cout<<"YESTERDAY = D1"<<endl;
33     d1.displayDate();
34
35     date today={6,"May",2026};
36
37     cout<<"TODAY"<<endl;
38     today.displayDate();
39
40     d1=today;
41     cout<<"D1 = TODAY"<<endl;
42     d1.displayDate();
43 }//end of main
44
```

STRUCTURES...

- In standard C: Functions are **not** allowed to be defined within a structure; Hold only data
- In C++ : Both Data & Functions can be declared in the structure.
- The members of a structure are visible to all functions that are within the scope of the structure (by Default members are PUBLIC).

ACTIVITY

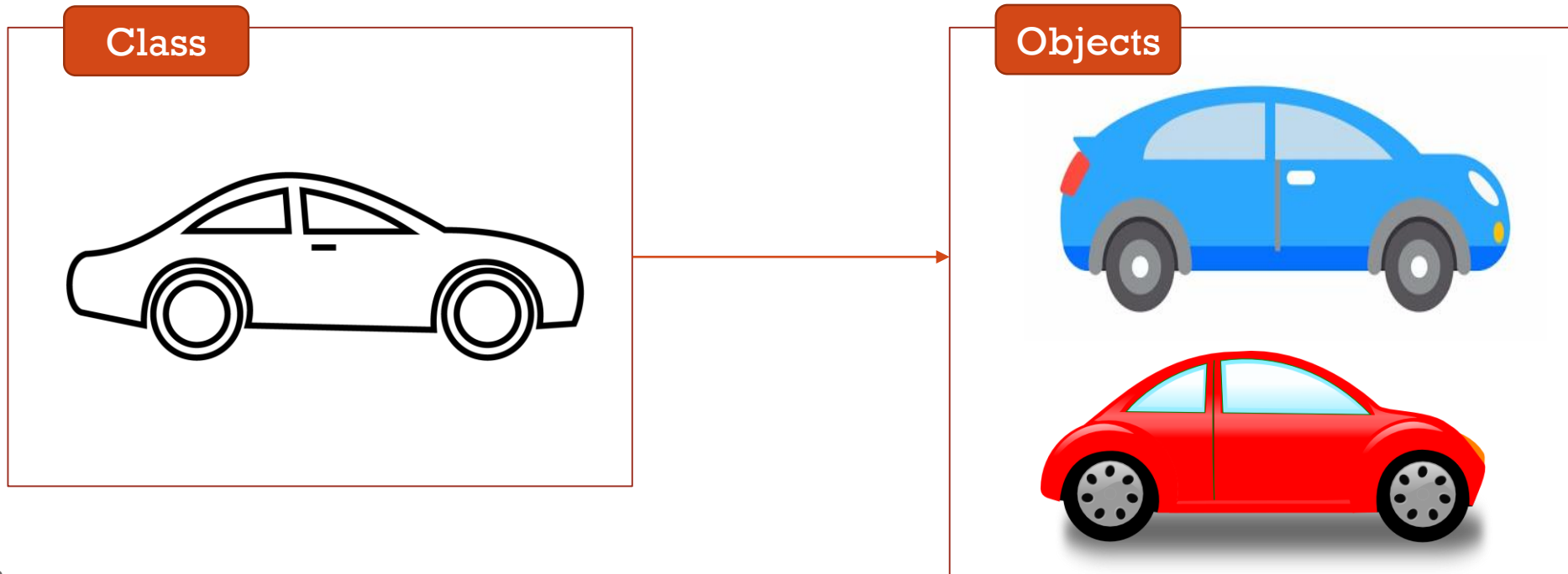
```
2  #include<string.h>
3  using namespace std;
4
5  //defining the structure date
6  struct date{
7      int day;
8      char month[10];
9      private:
10     int year=2024;
11
12     public:
13     void printYear()
14     {
15         cout<<"Year: "<<year<<endl;
16     }
17 }; // end of structure definition
18
19 int main()
20 {
21     date today;
22     today.day=28;
23     strcpy(today.month,"November");
24
25     cout<<"Day:"<<today.day
26         <<" Month:"<<today.month;
27     today.printYear();
28 }
```

STRUCTURES VS CLASSES

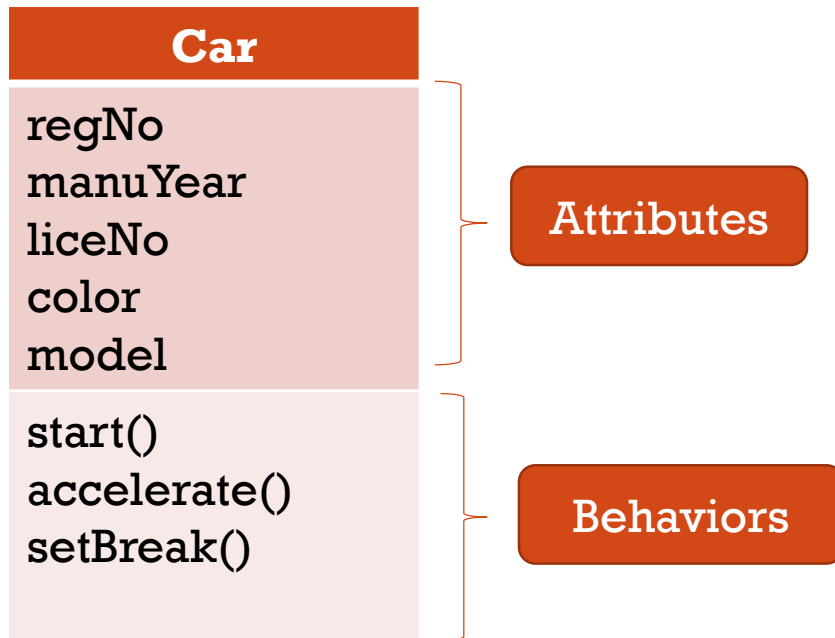
- Classes in C++ are a natural evolution of the C notion of structures.
- Both are user-defined data types.
- Both hold a bunch of different data types.
- A class is a user-defined blueprint or prototype from which objects are created.
- A class combines the fields and methods(member function which defines actions) into a single unit.
- The syntactical distinction between structures and classes in C++ is minimal...
- Classes in C++: by default members are PRIVATE.

WORKING WITH CLASSES

- Two main building blocks of Object Orientation.
- A class is a **template** for objects, and an object is an **instance** of a class.



CLASSES IN C++



class car

{

public:

Access specifier

int regNo;

int manuYear;

string liceNo;

string color;

string model;

Data Members

void start();

int accelerate();

void setBreak();

Member Functions

};

End of the class Definition

Class Body

CLASS MEMBERS

Functions that are declared within a class are called **member functions**

Variables that are declared within a class are called **data members**

CREATING OBJECTS

- An object is created from a class.
- To create an object, specify the class name, followed by the object name.
 - Example: `car myRedCar;`
- To access the class attributes/member functions, use the dot syntax (.) on the object.
 - Example: `myRedCar.regNo;`

```
int main()
{
    car myRedCar;
    car myBlueCar={456,2018,"CP345","Blue","B"};
}
```

DECLARE OBJECTS

Syntax:

```
ClassName ObjectName;
```

DOT OPERATOR

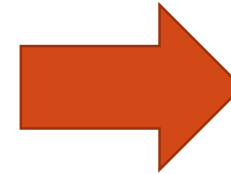
- Whenever you refer to a member of a class from code outside the class definition, you must prefix the member's name with the dot operator and the name of the object it belongs to.
- The member may be either data or a function.

`objectName.memberName`

`objectName.function()`

ACTIVITY

```
1  #include<iostream>
2  using namespace std;
3
4  class car
5  {
6      int regNo;
7      int manuYear;
8      string liceNo;
9      string color;
10     string model;
11
12     void start();
13     int accelerate();
14     void setbreak();
15 };
16
17 int main()
18 {
19     car myRedCar ={234,2017,"Wp4567","Red","C"};
20     car myBlueCar={456,2018,"CP345","Blue","B"};
21
22     cout<<"My Red Car: "<<myRedCar.liceNo<<" "<<endl;
23     cout<<"My Blue Car: " <<myBlueCar.liceNo<<" "<<endl;
24 }
```

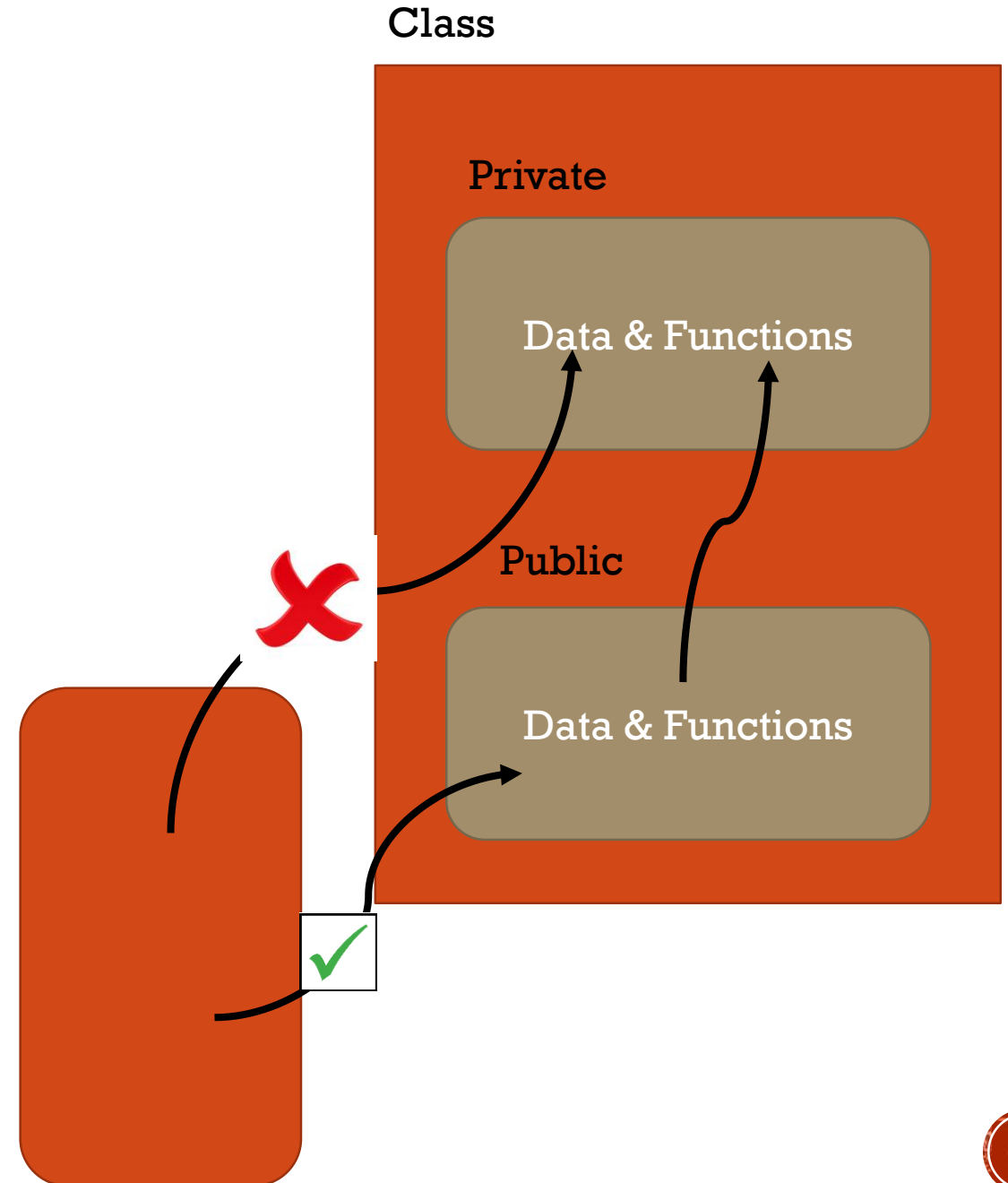


Compilation Error

The default access model for classes is private so all members after the class header and before the first member access specifier are private

ACCESS SPECIFIERS

- Define how the members (attributes and methods) of a class can be accessed.
- *Data hiding*: Key feature of object-oriented programming
 - The data is concealed/wrapped within a class so that it cannot be accessed mistakenly directly or by functions outside the class.
 - Private data or functions can only be accessed from within the class.
 - Public data or functions are accessible from outside the class.
- Hide data from parts of the program that don't need to access it.
- Data in one class is hidden from other classes.

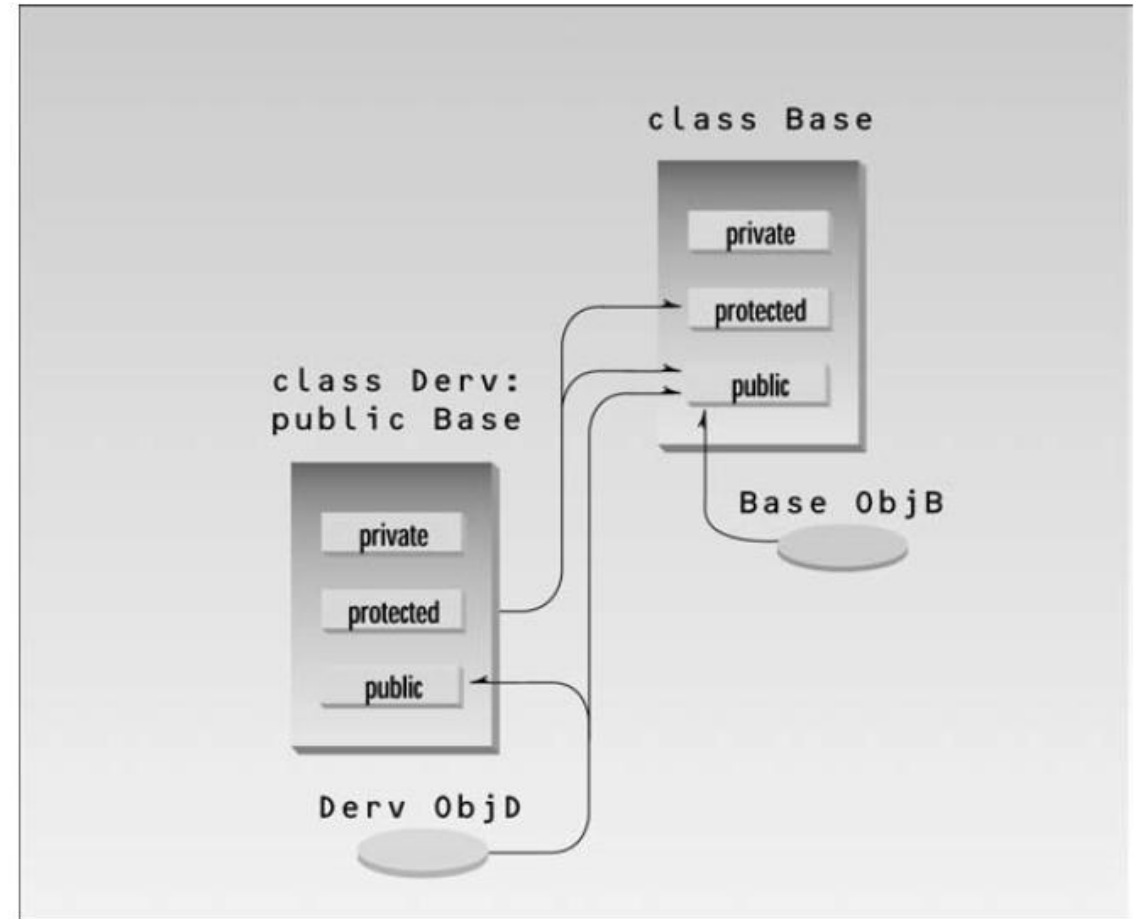
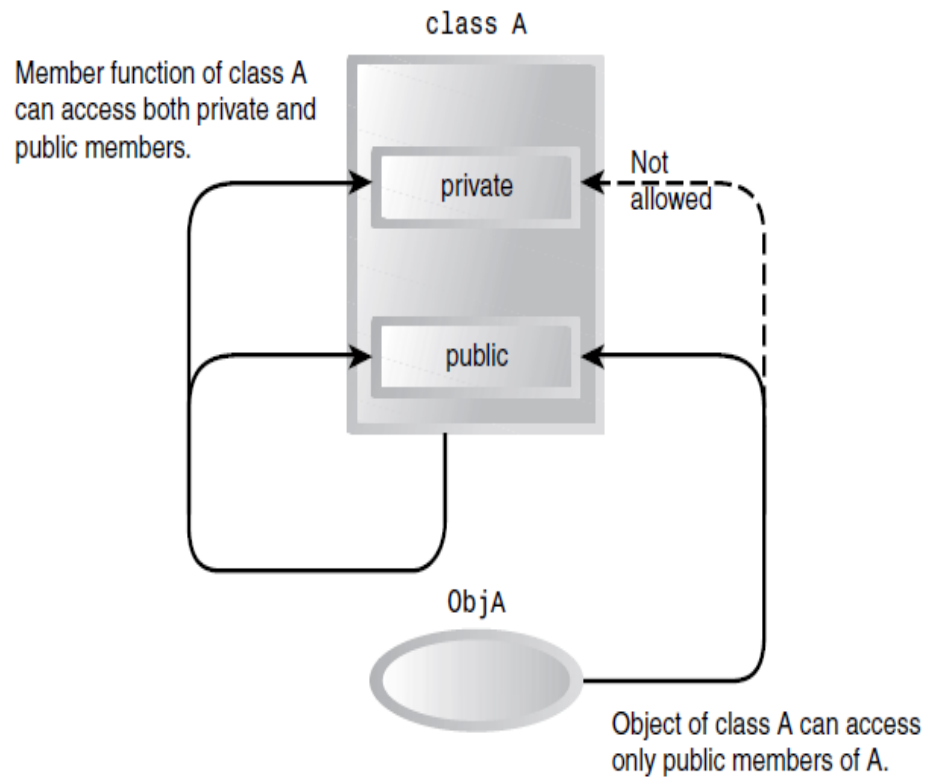


ACCESS SPECIFIERS

Keyword **public**:
makes subsequent
members public (can
be accessed by non-
member functions)

Keyword **private**:
makes subsequent
members private
(cannot be accessed
by non-member
functions)

Keyword **protected**:
Somewhat similar to
the keyword private.

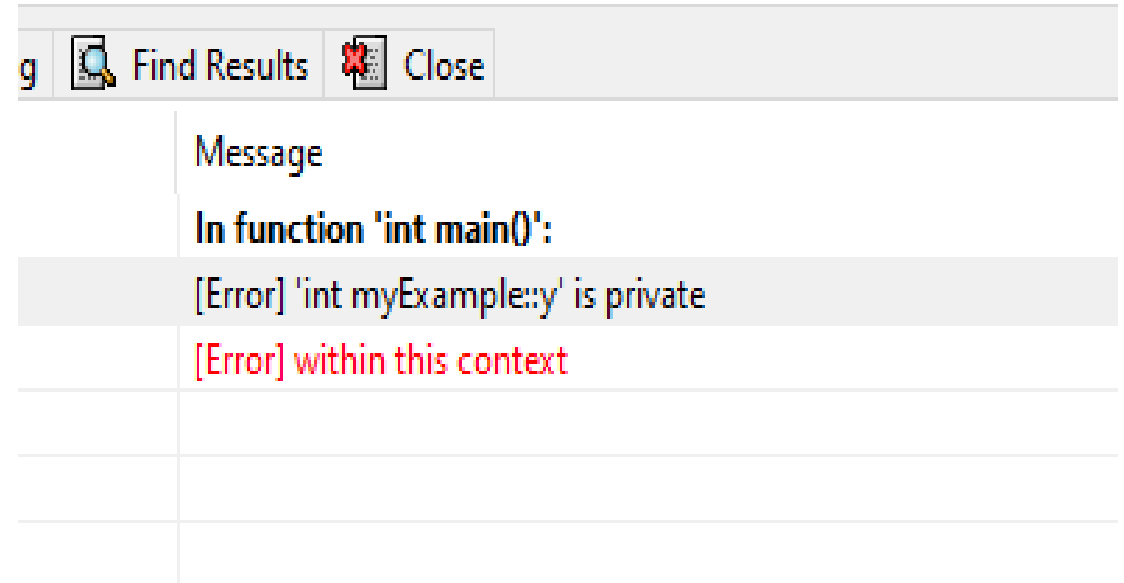


ACCESS SPECIFIERS-EXAMPLE

```
class myExample
{
    public: // Public access specifier
        int x; // Public attribute
    private: // Private access specifier
        int y; // Private attribute
};

int main() {
    myExample E1;
    E1.x = 3;
    E1.y = 4;
}
```

OUTPUT ?



DEFINING CLASSES

```
class ClassName{           //class Name
AccessSpecifier:          //public, private or protected
Datamembers;              //Variables/Attributes
MemberFunctions(){}       //Methods to access member functions/Behavior
////////////////////////
AccessSpecifier:          //public, private or protected
Datamembers;              //Variables/Attributes
MemberFunctions(){}       //Methods to access member functions/Behavior
////////////////////////
};                          //end of the class Name
```

ACTIVITY

- Define a class to create a new datatype called Date (Should include day, month and year)
- Define two functions called
 - setDate to initialize values by passing parameters
 - getDate to print the value (Date) on the console
- Create a variable called “today” of data type Date
- Set the value to 06/May/2026
- Display the value on the console

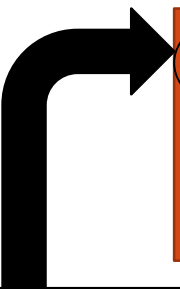
CLASS: DATE

```
1  #include<iostream>
2  using namespace std;
3
4  class date
5  {
6      int day;
7      string month;
8      int year;
9
10     void setDate(int D,string M,int Y){
11         day = D;
12         month = M;
13         year = Y;
14     }
15
16     void getDate(){
17         cout<<"Day:"<<day<<" Month:"<<month<<" Year:"<<year<<endl;
18     }
19 }; // end of class definition
20
```

CLASS AS A DATA TYPE

- Once a class has been declared, it can be used as object/array declarations

Example:



```
Date newDate,           // object of type Date  
    Date DtArray[10];    // array of Date objects  
    Date *newDatePtr;    // Object pointer
```

The class name becomes
the new type specifier.

DATA MEMBERS AND MEMBER FUNCTIONS

Accessing Data members / Member functions

- DOT (.) Operator
 - Example: `MyObject.variableX; MyObject.func();`
- Refer to a member of a class via a pointer to an object, (->) Operator

A pointer variable, or simply a pointer, contains the location, or address in the memory, of a memory cell

ACTIVITY

- Define a class to create a new datatype called Date (Should include day, month and year)
- Define two functions called
 - setDate to initialize values by passing parameters
 - getDate to print the value (Date) on the console
- Create a variable called “today” of data type Date
- Set the value to 06/May/2026
- Display the value on the console
- **Create a pointer variable called ptrDate to assign the value of “today”**
- **Display the value of ptrDate**

ACCESS OPERATOR (->): EXAMPLE

```
int main()
{
    date today;
    today.setDate(6,"May",2026);

    date *ptrDate;
    ptrDate=&today;

    cout<<"CHECK POINTER"<<endl;
    ptrDate->getDate();
}
```

Date today;



DATA MEMBERS AND MEMBER FUNCTIONS

Accessing Data members / member functions

- DOT (.) Operator
 - Example: `MyObject.variableX; MyObject.func();`
- Refer to a member of a class via a pointer to an object, (->) Operator
- Accessing a data member depends on the access modifier of the data member.